Progress Report: LensLab
Camera Calibration, GPU Undistortion, and Virtual Camera Matching in Unity

Julie Yang

Abstract

This report summarizes the current progress of LensLab, a computer vision course project that integrates classical camera calibration and pose estimation with a Unity-based rendering workflow. The project goal is to build a practical plugin-oriented pipeline that starts from offline ChArUco calibration images, exports shared camera parameters, performs GPU undistortion inside Unity, estimates camera pose from a known target, and ultimately supports virtual camera alignment. At the current milestone, the project has completed an offline end-to-end validation path from Python/OpenCV to Unity. Specifically, calibration, JSON-based parameter exchange, CPU/GPU undistortion validation, calibrated camera projection in Unity, and single-image ChArUco pose estimation have all been implemented and verified. The remaining work is focused on turning this validated prototype into a cleaner runtime workflow for virtual content anchoring and real-time matching.

1. Project Overview

LensLab is designed as a practical bridge between computer vision algorithms and a real-time graphics engine. The main motivation is to move beyond isolated OpenCV scripts and build a reproducible workflow that can be used inside Unity. The intended pipeline is:

Calibration images → calibration parameters → Unity loading → GPU undistortion → pose estimation → virtual camera or content alignment.

Compared with the original project idea, the main conceptual direction remains the same, but the implementation strategy has become more concrete and more engineering-driven. In particular, the project has shifted from a broad "camera matching plugin" idea to a staged vertical slice with explicit interfaces between Python and Unity. This change was necessary because the highest-risk parts of the project were not individual algorithms, but the system boundaries: parameter exchange, coordinate conventions, projection consistency, and validation inside Unity.

The current milestone therefore focuses on correctness and integration rather than runtime polish. The central question for this phase has been: can the project reliably move from OpenCV outputs to Unity-side visual validation without silently breaking assumptions about intrinsics, distortion, projection, or pose?

2. Tasks

1. Design the repository structure and define a stable project architecture for calibration, undistortion, pose estimation, and Unity integration.
2. Implement a Python/OpenCV calibration pipeline using a ChArUco board and export the result in a shared JSON format.
3. Build a CPU undistortion reference path and compare it against a Unity compute-shader GPU undistortion path.
4. Load calibration parameters inside Unity and map the estimated intrinsics to a calibrated Unity camera projection matrix.
5. Estimate single-image board pose in Python and export pose data in a Unity-readable format.
6. Validate the OpenCV-to-Unity pipeline by rendering a board-aligned region back onto the corresponding reference image in Unity.
7. Document the current milestone clearly so the next stage can focus on runtime anchoring and final demonstration.

3. Collaboration Strategy

The project has effectively been split into two interacting workflows: an OpenCV validation workflow in Python and a Unity integration workflow in C#. Python is used as the source of truth for calibration and pose mathematics, while Unity is treated as the target runtime environment. This separation has been especially important for debugging because it allows numerical correctness and rendering

correctness to be validated independently before they are merged.

The development strategy has been incremental. Rather than attempting live camera integration immediately, each subsystem was first validated in an offline setting: first calibration, then JSON export, then Unity-side loading, then GPU undistortion, then pose estimation, and finally board-region validation in Unity. This staged workflow reduced ambiguity whenever something failed, since there was always a narrower interface to inspect.

4. Proposed Approach

The current system is organized into four layers:

Calibration layer. Python/OpenCV scripts detect ChArUco corners, calibrate the camera, compute reprojection statistics, and export intrinsics and distortion coefficients. The current implementation supports ChArUco-driven calibration and produces a JSON file containing image resolution, focal lengths, principal point, distortion coefficients, board metadata, and reprojection summaries.

Undistortion layer. The undistortion stage was implemented twice: once as a CPU reference in Python/OpenCV and once as a compute-shader implementation in Unity. The goal was not simply to undistort an image, but to confirm that Unity-side GPU processing is consistent with a trusted CPU reference. This stage also exposed practical issues such as color space mismatches, import settings, and render texture handling.

Projection layer. After calibration parameters were loaded into Unity, they were converted into a calibrated camera projection matrix. This allowed the Unity camera to approximate the same imaging geometry as the real camera. This stage provided the geometric basis for later content alignment.

Pose layer. Pose estimation is currently implemented offline in Python using ChArUco detections and `solvePnP`. The resulting pose is exported to JSON and consumed in Unity. A Unity-side validation object then uses board-model information to render a semi-transparent surface onto the reference image, allowing visual confirmation that the estimated pose is broadly consistent with the detected target region.

This layered design was chosen because it mirrors the final plugin architecture. Instead of treating each computation as a separate script, the project is converging toward a single pipeline with explicit shared contracts.

5. Data

The current dataset is small but purposefully structured for validation rather than scale. The main sources are:

- ChArUco calibration images captured from a real camera.
- Reference images used to validate CPU and GPU undistortion.
- A ChArUco pose validation image (currently image 003) used to verify pose export and Unity-side alignment.

The calibration board configuration is stored explicitly in a YAML file and includes board type, grid dimensions, square size, marker size, and dictionary type. This has been important because the project repeatedly depended on exact agreement between physical board specification, OpenCV board construction, and Unity-side assumptions.

At this stage the project does not depend on a large benchmark dataset. The emphasis is on trustworthy geometric validation using internally generated data. For a future stage, the dataset could be extended with short image sequences or video frames for temporal pose stability experiments.

6. Initial Results

The most important result of the current milestone is that the full offline chain from Python to Unity is now operational.

Calibration. The calibration pipeline successfully detects ChArUco boards, filters usable frames, estimates intrinsics and distortion parameters, and exports calibration results as JSON. Reprojection statistics are also written out so the calibration can be evaluated numerically rather than only visually.

Undistortion. CPU undistortion previews were generated in Python and compared against Unity GPU undistortion. Early issues such as color-space mismatches and texture import distortions were identified and corrected. After those corrections, the GPU output became visually consistent with the CPU reference, which is an important integration result.

Projection mapping. Unity successfully loads the calibration JSON and applies a calibrated projection matrix to the main camera. This means the virtual camera is no longer using only Unity's default field-of-view model, but is instead being driven by estimated real-camera intrinsics.

Pose estimation. Single-image ChArUco pose estimation was successfully implemented in Python. A representative pose output for image 003 includes low reprojection error and plausible translation values. Unity can now load this pose and render a corresponding validation surface back onto the target image.

Board-region validation. The current board-region overlay in Unity is not yet intended as the final visualization, but it confirms that the pose and projection chain is functioning well enough to place a virtual board region back onto the ChArUco target area. This is the strongest evi-

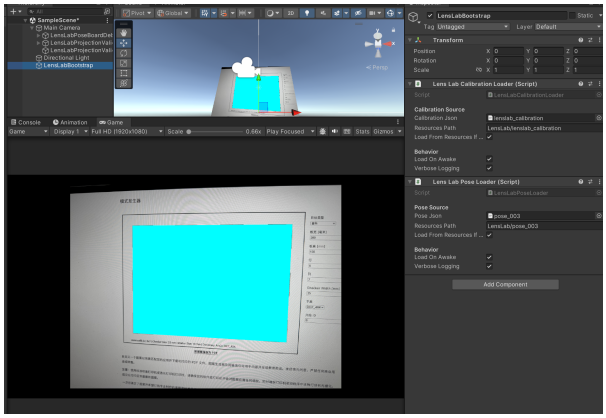

Figure 1. Example pose-estimation validation output for ChArUco image 003. Detected board corners and coordinate axes are rendered using the calibrated OpenCV pipeline before the pose result is consumed by Unity.

dence so far that the OpenCV-to-Unity geometric pipeline is working as intended.

7. Current Concerns and Questions

Although the current milestone is successful, several concerns remain and define the next phase of work.

1. **Runtime readiness.** The current system is still an offline validation prototype. The next challenge is to adapt the same pipeline to a runtime setting with live image input.

2. **Pose stability.** Current pose validation is based on single images. The project still needs temporal pose tracking, smoothing, and robustness analysis across multiple frames.

3. **Content anchoring.** The current Unity validation uses a debug board region. The next step is to replace this with virtual content anchored in board space so that the system begins to resemble the final plugin goal.

4. **Plugin packaging.** The code is functionally organized, but it has not yet been packaged as a polished user-facing Unity plugin with a minimal inspector workflow.

5. **Evaluation scope.** Future work should better distinguish between geometric correctness, visual quality, and runtime usability, since all three matter but require different validation strategies.

Overall, the main risk at the start of this project was whether OpenCV-side calibration and pose estimation could be brought into Unity without breaking the geometry. The current milestone suggests that this risk has been substantially reduced. The next milestone will therefore focus less on proving the math path and more on transforming that validated path into a usable demonstration of virtual content alignment.